데이터베이스(SQL)

2장. 데이터베이스 · PostgreSQL 이해

학습 목표

1) 데이터와 정보를 구분하고, 데이터를 구조에 따라 정형 · 반정형 · 비정형으로 분류한다.
2) 파일 관리의 한계와 데이터베이스 · DBMS의 차이를 설명한다.
3) 관계형 데이터베이스의 구조를 이해한다.
4) DBMS의 종류를 비교하고 PostgreSQL의 특징을 설명한다.
5) SQL 명령어 분류(DDL · DML · DCL · TCL)와 CRUD를 이해하고 기본 예제를 실행한다.

전체 흐름

1) 데이터 → 2) 정보 → 3) 데이터베이스와 DBMS → 4) 관계형 데이터베이스(RDBMS)

5) PostgreSQL → 6) SQL

01 데이터와 정보

데이터의 정의와 유형

데이터(Data)는 관찰, 측정, 기록을 통해 수집한 **사실이나 값**

데이터 유형	예시
정수	1000
실수	85.3
문자열	M001
날짜	2025-01-01
날짜와 시간	2025-01-01 09:30:00
논리값	true, false
문서, 이미지	점검 보고서, 불량 사진

Date	Product	Batch	Operator	Start Time	End Time
2024-08-29	OR-600	422111	Mac	11:50:00	14:05:00
2024-08-29	LE-600	422112	Mac	14:05:00	15:45:00
2024-08-29	LE-600	422113	Mac	15:45:00	17:35:00
2024-08-29	LE-600	422114	Mac	17:35:00	19:15:00
2024-08-29	LE-600	422115	Charlie	19:15:00	20:39:00
2024-08-29	LE-600	422116	Charlie	20:39:00	21:39:00
2024-08-29	LE-600	422117	Charlie	21:39:00	22:54:00
2024-08-30	CO-600	422118	Dee	04:05:00	06:05:00
2024-08-30	CO-600	422119	Dee	06:05:00	07:30:00
2024-08-30	CO-600	422120	Dee	07:30:00	09:22:00

공식 문서: 자료형 전체 목록(18 버전 원문)

데이터의 주요 특성

특성	설명
사실성	실제로 관찰하거나 측정한 사실을 표현한다
표현 가능성	같은 대상을 여러 형식(문자열, 날짜형 등)으로 표현할 수 있다
지속성	업무가 수행되는 동안 계속 생성되고 누적된다
품질 의존성	잘못 입력되면 분석 결과도 잘못된다

정보의 정의

정보(Information)는 데이터를 분석하거나 해석하여 **의미를 부여한 결과**

불량률 = 불량수량 / 총생산량 × 100
= 30 / 1,000 × 100
= 3%

데이터
생산량 1,000개, 불량 30개

→

정보
"불량률은 3%이다"

데이터와 정보의 차이

구분	데이터	정보
의미	관찰하거나 기록한 값	데이터를 처리하여 얻은 의미
상태	분석 전 원재료	분석 후 결과
활용	저장과 처리의 대상	판단과 의사결정의 근거

구조에 따른 데이터 분류

구분	구조	예시	SQL 직접 처리
정형 데이터	고정된 행과 열	테이블, Excel,구조화된 CSV	용이함
반정형 데이터	키-값, 태그, 계층 구조	JSON, XML	DBMS 기능에 따라 가능
비정형 데이터	고정된 구조 없음	이미지, 음성, 영상, 문서	내용 분석에는 별도 기술 필요

정형 · 테이블

id	온도	상태
M001	85.3	정상

모든 행이 같은 열 구조

반정형 · JSON

```
{  "id": "M001",
   "temp": 85.3  }
```

키와 값으로 구조 표현

비정형 · 사진/음성




파일 내용이 표 구조는 아님

정형 데이터

열 이름, 자료형, 제약조건 등 **구조가 미리 정의된** 데이터

machine_id	production_date	output_qty	defect_qty
M001	2025-01-01	1,000	15
M002	2025-01-01	850	22

1) 검색, 정렬, 집계, 조인이 용이하다

2) 자료형과 제약조건을 적용할 수 있다

가로 한 줄 → 생산 기록 1건

세로 한 줄 ↓ 같은 종류의 값

반정형 데이터

고정된 열 구조는 없지만 내부에 **키-값 쌍, 태그, 계층 구조**(데이터 안에 데이터가 중첩되는 구조)가 존재하는 데이터


```
{
"machine_id": "M001",
"timestamp": "2025-01-01T09:00:00",
"temperature": 85.3
}
```


"machine_id" : "M001"

키 (이름) **값 (내용)**

공식 문서: JSON 자료형(18 버전 원문) / COPY(CSV 적재, 18 버전 원문)

CSV의 분류

주의: CSV 분류

"JSON, XML, CSV는 모두 반정형 " 이라고 한다. **일정한 행과 열 구조를 가진 CSV는 일반적으로 정형 데이터로 처리**한다. 단, CSV 자체에는 자료형과 제약조건이 없으므로 적재 시 별도로 정의해야 한다.

비정형 데이터

행과 열로 직접 표현하기 어려운 데이터 (불량 사진, 작동 영상, 음성 메모, 계약서 PDF 등)

1) 파일명·생성일·저장 경로 같은 **메타데이터**로 관리 가능

2) 내용 분석은 텍스트 검색·컴퓨터 비전 등 기술 활용

유형별 분석 기술

유형	예시	분석 기술
텍스트	이메일, 웹 페이지, SNS 게시물	자연어 처리(NLP)
이미지 · 비디오	불량 사진, 작동 영상	컴퓨터 비전
오디오	음성 메모, 상담 녹취	음성 인식



불량 사진



설비 영상



음성 녹음



문서 파일

* 교육용 AI 생성 예시 이미지

비정형 데이터는 어디에 저장할까?

예: 공장의 불량 사진 — 파일 저장과 정보 관리를 나누어 생각해 보기



① 불량 사진 촬영

defect001.jpg

→

② 파일 저장소

AWS S3

사진 파일 자체 저장

→

③ 파일 정보 관리

PostgreSQL

설비번호 · 촬영일 · S3 위치

파일은 S3 등에 저장하고 PostgreSQL에는 파일 위치와 메타데이터를 저장.

센서 데이터의 분류

주의: 센서 데이터의 분류

센서 데이터를 비정형으로 분류하는 자료도 있으나, 측정 시각 · 설비번호 · 측정값처럼 **일정한 구조를 가진 센서 데이터는 정형 데이터로 처리하는 경우가 많다**.

측정 시각	설비번호	측정값
datetime	**machineid**	**volt, rotate, pressure, vibration**

메타데이터

메타데이터(Metadata)는 **데이터를 설명하는 데이터**

1) 예: "output_qty의 자료형은 정수이다", "output_qty에는 음수를 저장할 수 없다"

2) 관계형 DB에서는 테이블명, 열 이름, 자료형, 키, 제약조건이 메타데이터에 해당

https://dataon.kisti.re.kr/data_mgnt_guideline_09.do

순번	용어	설명
1	contributor	An entity responsible for making contributions to the resource
2	coverage	The spatial or temporal topic of the resource, the spatial applicability of the resource, or the jurisdiction under which the resource is relevant
3	creator	An entity primarily responsible for making the resource
4	date	A point or period of time associated with an event in the lifecycle of the resource
5	description	An account of the resource
6	format	The file format, physical medium, or dimensions of the resource
7	identifier	An unambiguous reference to the resource within a given context
8	language	A language of the resource
9	publisher	An entity responsible for making the resource available
10	relation	A related resource

02 데이터베이스와 DBMS

파일 관리의 한계

한계	설명
데이터 중복	여러 파일에 같은 고객 정보가 반복 저장된다
데이터 불일치	일부 파일만 수정하면 값이 서로 달라진다
동시 작업	여러 사용자가 같은 파일을 수정하면 충돌한다
무결성	"주문번호 중복 금지" 같은 규칙을 강제할 수 없다
권한 관리	사용자별 조회/수정 권한을 구분하기 어렵다
백업과 복구	손상 시 복구 시점 관리가 어렵다

무결성(Integrity)

데이터가 정해진 규칙을 벗어나지 않고 **정확하고 일관되게 유지되는 성질**

"생산량은 음수일 수 없다"

"존재하지 않는 고객의 주문은 있을 수 없다"

데이터베이스와 DBMS

데이터베이스(Database): 관련된 데이터를 일정한 구조에 따라 저장한 **데이터 집합**

DBMS(Database Management System): 데이터베이스를 생성하고 관리하는 **소프트웨어**

구분	데이터베이스	DBMS
의미	일정한 구조(스키마)에 따라 저장된 데이터의 집합	데이터베이스를 생성 · 관리하는 소프트웨어
역할	데이터를 담고 있음	데이터를 저장 · 조회 · 변경하고 보호함
예시	고객, 주문, 제품 데이터	PostgreSQL, MySQL, SQL Server
비유	도서관의 책과 자료	도서관 관리 시스템(사서의 업무 시스템)

공식 문서: 기본 구조(한국어 번역, 13 버전 기준) / Architectural Fundamentals(18 버전 원문)

DBMS의 주요 기능

테이블 생성	데이터 저장 · 조회 · 변경	**제약조건 관리**	권한 관리
동시성 제어	**트랜잭션**	인덱스	**백업과 복구**

핵심 관리 기능

1) 무결성 관리

Integrity

제약조건으로 규칙에 어긋나는 데이터의 입력·변경을 막아, 정확성과 일관성을 유지한다.

관련 기능 제약조건 관리

2) 동시성 관리

Concurrency Control

여러 트랜잭션이 같은 데이터에 동시에 접근해도, 작업이 서로 꼬이지 않도록 제어한다.

관련 기능 동시성 제어 · 트랜잭션

3) 복구 관리

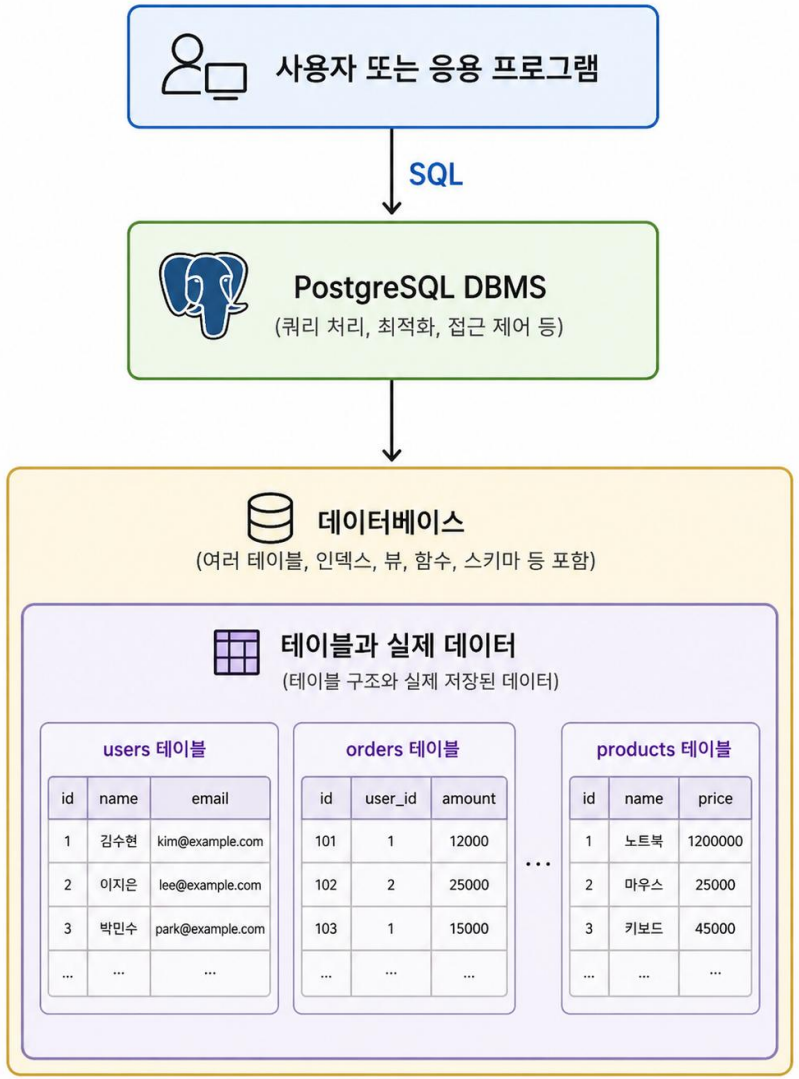
Recovery

장애가 나도 변경 기록(로그)으로, 커밋된 데이터를 장애 이전의 일관된 상태로 되살린다.

관련 기능 트랜잭션 · 백업과 복구

공식 문서: Constraints(18 버전 원문) · Concurrency Control(18 버전 원문) · Write-Ahead Logging(18 버전 원문) · Backup and Restore(18 버전 원문)

DBMS · 데이터베이스


사용자 또는 응용 프로그램
SQL
PostgreSQL DBMS
(쿼리 처리, 최적화, 접근 제어 등)
데이터베이스
(여러 테이블, 인덱스, 뷰, 함수, 스키마 등 포함)
테이블과 실제 데이터
(테이블 구조와 실제 저장된 데이터)
users 테이블
id name email
1 김수현 kim@example.com
2 이지은 lee@example.com
3 박민수 park@example.com
... ...
orders 테이블
id user_id amount
101 1 12000
102 2 25000
103 1 15000
... ...
...
products 테이블
id name price
1 노트북 1200000
2 마우스 25000
3 키보드 45000
... ...

03 관계형 데이터베이스

관계형 데이터베이스의 기본 구조

관계형 데이터베이스는 데이터를 **관계(테이블)** 형태로 관리하는 데이터베이스

용어	다른 이름	의미
테이블	관계(Relation)	행과 열로 구성된 저장 구조, 테이블은 데이터를 저장하는 기본 단위
행	레코드, 튜플	하나의 대상(고객 한 명, 주문 한 건)에 대한 데이터 묶음
열	컬럼, 속성	대상이 가진 속성(고객명, 도시 등)

행(Row)

행은 하나의 데이터 레코드를 의미한다.

용어	사용 맥락
Row(행)	일반적인 표현
Record(레코드)	데이터 저장 관점
Tuple(튜플)	이론/학술 관점

열(Column)

열은 데이터의 속성을 의미한다.

용어	사용 맥락
Column(열)	일반적인 표현
Field(필드)	데이터 입력 관점
Attribute(속성)	이론/설계 관점

행과 열의 의미

customer_id	customer_name	city
C001	김민수	서울
C002	이서연	부산

행 하나 = 고객 한 명

열 하나 = 고객이 가진 속성 하나

→ 가로(Row): 고객 정보 한 건

↓ 세로(Column): 도시 속성

하나의 테이블에 모두 저장한 경우

고객 정보 주문 정보 제품 정보 생산 정보 품질 정보

고객명	지역	주문번호	주문일자	제품명	단가
김철수	서울	O001	2025-01-01	볼트	500
김철수	서울	O002	2025-01-05	너트	300
김철수	서울	O003	2025-01-10	볼트	500

하나의 테이블에서 발생하는 문제

문제	설명
데이터 중복	김철수, 서울이 계속 반복 저장된다
저장 공간 낭비	같은 값이 여러 번 저장된다
수정 이상	김철수가 부산으로 이사하면 모든 행을 수정해야 한다
삽입 이상	주문하지 않은 고객은 등록할 수 없다
삭제 이상	주문을 삭제하면 고객 정보도 사라진다

테이블 분리로 해결하기

데이터를 여러 테이블로 나누어 저장하고, 필요할 때 연결해서 사용한다.

고객 테이블

고객ID	고객명	지역
C001	김철수	서울

주문 테이블

주문번호	고객ID	주문일자
O001	C001	2025-01-01
O002	C001	2025-01-05

관계형 데이터베이스

관계형 데이터베이스는 데이터를 **행과 열로 이루어진 테이블(관계)** 로 표현하고, **키(Key)와 제약조건(Constraint)** 을 이용해 데이터의 관계와 무결성을 관리하는 **데이터베이스 시스템(RDBMS)**이다. 테이블을 적절히 나누는 정규화는 관계형 데이터베이스를 설계하는 대표적인 방법 중 하나다.

기본키(Primary Key)

한 테이블의 필드(열) 또는 필드의 조합 중, 각 행을 유일하게 식별할 수 있는 키

1) 중복 불가, **NULL 불가**

2) 예: 고객 테이블의 customer_id

NULL

값이 없음(미입력, 알 수 없음)을 나타내는 특별한 상태. 숫자 0이나 빈 문자열과는 다르다. 기본키가 NULL이면 그 행이 누구인지 식별할 수 없으므로 허용하지 않는다.

PK는 테이블당 하나지만, 여러 열을 합쳐 하나의 PK로 만들 수 있다. => 복합키(Composite Key).

공식 문서: 기본키(18 버전 원문)

복합키가 필요한 이유

주문 하나에 여러 제품이 들어갈 수 있다.
order_id만으로는 각 행을 구분할 수 없고, product_id만으로도 구분할 수 없다. 두 개를 합쳐야 유일해진다.

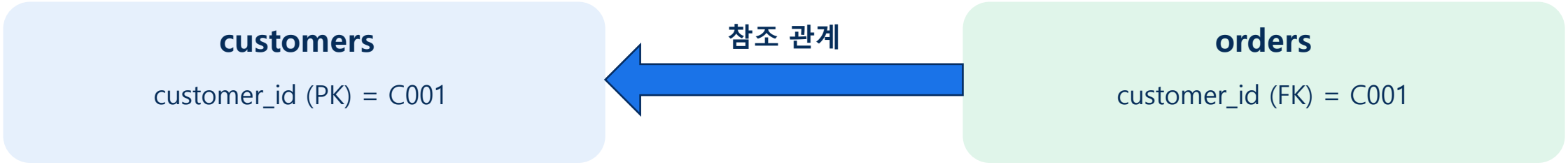
order_id	product_id	quantity
10248	11	12
10248	42	10
10249	14	9

외래키(Foreign Key)

한 테이블의 필드(열) 또는 필드의 조합 중, 다른 테이블의 행을 식별하는 키(주로 기본키)를 참조하여 테이블 간 관계를 나타내는 키

1) 존재하지 않는 고객의 주문 등록을 방지한다 → **참조 무결성**

2) 고객 1명 : 주문 여러 건 → **일대다 관계**



고객 1명에 주문 여러 건 연결 가능

공식 문서: 외래키(18 버전 원문)

참조 무결성

참조 무결성: FK가 가리키는 값이 실제로 존재하도록 하여 데이터 간 관계의 일관성을 유지하는 원칙.

```
-- customers에 'ZZZZZ'가 없으면 오류 발생
INSERT INTO orders (order_id, customer_id, order_date)
VALUES (30000, 'ZZZZZ', DATE '2025-01-01');
```

오류: "orders" 테이블에서 자료 추가, 갱신 작업이 "fk_orders_customers" 참조키(foreign key) 제약 조건을 위배했습니다

부모 테이블과 자식 테이블

구분	뜻	예
부모 테이블	참조되는 테이블 (일반적으로 PK 제공)	customers
자식 테이블	참조하는 테이블 (FK 보유)	orders

PK와 FK 비교

항목	Primary Key	Foreign Key
목적	행 식별	테이블 연결
중복	불가	같은 값을 여러 행에서 사용할 수 있음
NULL	불가	설정에 따라 허용
개수	테이블당 1개	여러 개 가능

관계형 DB를 사용하는 이유

장점	설명
구조 정의	열 이름과 자료형을 명확하게 정의한다
무결성	기본키, 외래키, CHECK 등으로 잘못된 입력을 차단한다
관계 관리	테이블을 분리하고 필요할 때 조인으로 결합한다
중복 감소	반복 정보를 별도 테이블로 분리한다
동시 작업	여러 사용자의 동시 조회 · 변경을 관리한다
트랜잭션	여러 작업을 하나의 단위로 처리한다

04 DBMS 종류와 PostgreSQL

DBMS의 유형

구분	정의	대표 제품	대표 사용 사례
관계형(RDBMS)	데이터를 행과 열로 된 테이블에 저장하고, 미리 정한 스키마에 맞춰 SQL로 다루는 DBMS	PostgreSQL, MySQL, Oracle	주문, 회계, 생산, 인사
문서형(NoSQL)	JSON 형태의 문서 단위로 데이터를 저장하는 DBMS	MongoDB	콘텐츠, 상품 카탈로그
키-값형(NoSQL)	키 하나에 값 하나를 짝지어 저장하는 DBMS	Redis	캐시, 세션

PostgreSQL이란?

PostgreSQL은 UC Berkeley의 POSTGRES 프로젝트에서 발전한 **객체-관계형 DBMS(ORDBMS)**

용어	설명
RDBMS	관계형 모델과 테이블 중심으로 관리
ORDBMS	관계형 모델 + 사용자 정의 자료형, 확장 기능

관계형 DBMS라고 불러도 틀리지 않지만, 확장 기능까지 포함하면 **객체-관계형 DBMS**가 더 정확한 표현이다.

공식 문서: PostgreSQL이 무엇인가?(한국어 번역, 13 버전 기준) / What Is PostgreSQL?(18 버전 원문)

05 SQL

SQL이란?

SQL(Structured Query Language)은 관계형 데이터베이스의 **구조와 데이터를 정의하고 처리하는 언어**

공식 문서: SQL 언어(한국어 번역, 13 버전 기준) / The SQL Language(18 버전 원문)

SQL 명령어 분류

구분	명칭	역할	예
DDL	Data Definition Language	객체 정의	CREATE, ALTER, DROP
DML	Data Manipulation Language	데이터 조회와 변경	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	권한 관리	GRANT, REVOKE
TCL	Transaction Control Language	트랜잭션 관리	COMMIT, ROLLBACK, SAVEPOINT

CRUD와 DML

CRUD	뜻	SQL
C (Create)	생성	INSERT
R (Read)	조회	SELECT
U (Update)	수정	UPDATE
D (Delete)	삭제	DELETE

조회 (SELECT)

```
SELECT customer_id, company_name, city
FROM customers
WHERE city = 'London';
```

테이블 생성 (DDL)

생산 이력 테이블 만들어보기

```
-- 확인하기: 같은 실습을 다시 실행할 수 있도록 기존 예제 테이블이 있으면 삭제한다.
DROP TABLE IF EXISTS production_logs;

CREATE TABLE production_logs (
production_id BIGINT GENERATED ALWAYS AS IDENTITY, -- 자동 증가 번호
machine_id VARCHAR(10) NOT NULL,
production_date DATE NOT NULL,
output_qty INTEGER NOT NULL,
defect_qty INTEGER NOT NULL,
CONSTRAINT pk_production_logs PRIMARY KEY (production_id),
CONSTRAINT ck_output_qty CHECK (output_qty >= 0),
CONSTRAINT ck_defect_qty CHECK (defect_qty >= 0),
CONSTRAINT ck_defect_output CHECK (defect_qty <= output_qty)
);
```

GENERATED ALWAYS AS IDENTITY:행이 추가될 때마다 DBMS가 **번호를 자동 생성**한다. 중간 번호는 비어 있을 수 있다.

추가 (INSERT)

```
INSERT INTO production_logs (machine_id, production_date, output_qty, defect_qty)
VALUES
('M001', DATE '2025-01-01', 1000, 15),
('M002', DATE '2025-01-01', 850, 22);
```

수정 (UPDATE)

```
UPDATE production_logs
SET defect_qty = 12
WHERE machine_id = 'M001'
AND production_date = DATE '2025-01-01';
```

삭제 (DELETE)

```
DELETE FROM production_logs
WHERE machine_id = 'M002'
AND production_date = DATE '2025-01-01';
```

트랜잭션과 ACID

트랜잭션(Transaction)은 여러 데이터 변경 작업을 **하나의 논리적 작업 단위**로 묶은 것. 관계형 DBMS의 트랜잭션은 일반적으로 **ACID** 성질로 설명한다.

성질	의미
원자성(Atomicity)	트랜잭션 내 작업은 전부 성공하거나 전부 취소된다
일관성(Consistency)	트랜잭션 전후로 데이터가 규칙(제약조건)을 만족한다
고립성(Isolation)	동시에 실행되는 트랜잭션이 서로 간섭하지 않는다
지속성(Durability)	확정(COMMIT)된 변경은 장애가 발생해도 보존된다

예: 계좌 이체에서 출금과 입금은 **함께 성공하거나 함께 취소**되어야 한다 (원자성)

공식 문서: 트랜잭션(한국어 번역, 13 버전 기준) / Transactions(18 버전 원문) / 동시성 제어 소개(한국어 번역) / Concurrency Control Introduction(18 원문)

트랜잭션 (TCL)

```
-- 두 UPDATE가 하나의 작업 단위로 묶이는 모습을 확인한다.
-- 원래 값을 유지하기 위해 마지막에 ROLLBACK한다.
BEGIN;

UPDATE production_logs
SET output_qty = output_qty - 10
WHERE machine_id = 'M001'
AND production_date = DATE '2025-01-01';



UPDATE production_logs
SET output_qty = output_qty + 10
WHERE machine_id = 'M002'
AND production_date = DATE '2025-01-01';

ROLLBACK;
```

SQL 작성 시 기본 원칙

원칙	예시
예약어는 대문자 (관례)	SELECT * FROM customers;
의미 있는 이름	temp (X) → process_temperature_k (O)
문자열은 작은따옴표	WHERE city = 'London'
날짜는 날짜 리터럴	WHERE production_date = DATE '2025-01-01'

공식 문서: 어휘 구조